CS 440 – Operating Systems
Project 3 – CPU Scheduling Simulation
50 (+5 extra credit) points available
Due in Canvas by Saturday, March 14, 2026 @ 11:59 PM

This assignment may be completed individually OR in pairs.

You may use ANY of the following programming languages: C, C#, C++, Java, Python. See me if you plan to use another language other than these five. There is NO bonus for using a particular language for this assignment.

1. Overview

You must implement FIVE CPU scheduling algorithms on the SAME generated process set:
1) First-Come, First-Served (FCFS – non-preemptive)
2) Shortest Job First (SJF – non-preemptive)
3) Shortest Remaining Time First (SRTF – preemptive)
4) Round Robin (RR – preemptive)
5) Random Selection (non-preemptive)

For EACH algorithm you must:
- Print a timestamped execution trace (see examples)
- Print total completion time
- Compute average waiting time
- Compute average response time

After all five algorithms, compute and print a deterministic CHECKSUM.

2. Inputs

Your program must prompt for the following INTEGER inputs:
- Random seed
- Number of processes n (2–10)
- Last arrival time A (0–99)
- Maximum burst time B (1–100)
- Round Robin quantum q (1–100)
- Context-switch latency L (0–10)

If input is invalid, print an error and re-prompt.

Example Input Prompts

```
Enter random seed: 42
Enter number of processes (2-10): 3
Enter last arrival time (0-99): 8
```

```
Enter maximum burst time (1-100): 10
Enter RR quantum (1-100): 3
Enter context-switch latency (0-10): 1
```

3. Scheduling Rules

General Rules:

- Time starts at 0.
- If no process is ready, jump time directly to next arrival (do NOT increment one cycle at a time).
- Context-switch latency L applies ONLY when switching between processes.
- Do NOT charge latency before the first process or after the final process.

SRTF Rule:

- Always run the process with smallest remaining time.
- Preempt ONLY if newly arrived process has STRICTLY smaller remaining time.
- If remaining times are equal, do NOT preempt.

4. Metric Definitions

For each process Pi:

- Response time(i) = first_start(i) − arrival(i)
- Waiting time(i) = completion(i) − arrival(i) − burst(i)

Averages:
avg_response = (sum of response times) / n
avg_wait = (sum of waiting times) / n

Important: response time is measured from the FIRST time a process gets the CPU. Do not overwrite it after preemption.

5. FULL Worked Example (All Five Algorithms + Step-by-Step Metrics)

The example below uses a fixed process set (for illustration). Your program will generate processes randomly, but your output must follow the same rules.

Example Process Set

```
Enter random seed: 152
Enter number of processes (2-10): 3
Enter last arrival time (0-99): 2
Enter maximum burst time (1-100): 6
Enter RR quantum (1-100): 2
Enter context-switch latency (0-10): 1
```

Processes (display this information above all runs):

```
P1: arrival=0, burst=5
P2: arrival=1, burst=3
P3: arrival=2, burst=6
Context-switch latency L=1
RR quantum q=2
```

5.1 FCFS (Non-preemptive)

FCFS:

```
@t=0, P1 selected for 5 units
@t=5, context switch 1 occurs
@t=6, P2 selected for 3 units
@t=9, context switch 2 occurs
@t=10, P3 selected for 6 units
@t=16, all processes complete
Completed in 16 cycles.
```

Step-by-step metric calculations (using the formulas in Section 4):

PID	Arrival	Burst	First Start	Completion	Response	Waiting
P1	0	5	0	5	0	0
P2	1	3	6	9	5	5
P3	2	6	10	16	8	8

```
Average response time = (0 + 5 + 8) / 3 = 4.33

Average waiting time  = (0 + 5 + 8) / 3 = 4.33
```

5.2 SJF (Non-preemptive)

```
SJF:
@t=0, P1 selected for 5 units
@t=5, context switch 1 occurs
@t=6, P2 selected for 3 units
@t=9, context switch 2 occurs
@t=10, P3 selected for 6 units
@t=16, all processes complete
Completed in 16 cycles.
```

Step-by-step metric calculations (using the formulas in Section 4):

PID	Arrival	Burst	First Start	Completion	Response	Waiting
P1	0	5	0	5	0	0
P2	1	3	6	9	5	5
P3	2	6	10	16	8	8

```
Average response time = (0 + 5 + 8) / 3 = 4.33

Average waiting time  = (0 + 5 + 8) / 3 = 4.33
```

5.3 SRTF (Preemptive)

```
SRTF:
@t=0, P1 selected for 1 unit
@t=1, context switch 1 occurs
@t=2, P2 selected for 3 units
@t=5, context switch 2 occurs
@t=6, P1 selected for 4 units
@t=10, context switch 3 occurs
@t=11, P3 selected for 6 units
@t=17, all processes complete
Completed in 17 cycles.
```

Step-by-step metric calculations (using the formulas in Section 4):

PID	Arrival	Burst	First Start	Completion	Response	Waiting
P1	0	5	0	10	0	5
P2	1	3	2	5	1	1
P3	2	6	11	17	9	9

```
Average response time = (0 + 1 + 9) / 3 = 3.33

Average waiting time  = (5 + 1 + 9) / 3 = 5.00
```

5.4 Round Robin (Preemptive)

```
RR (q=2):
@t=0, P1 selected for 2 units
@t=2, context switch 1 occurs
@t=3, P2 selected for 2 units
@t=5, context switch 2 occurs
@t=6, P3 selected for 2 units
@t=8, context switch 3 occurs
@t=9, P1 selected for 2 units
```

```
@t=11, context switch 4 occurs
@t=12, P2 selected for 1 unit
@t=13, context switch 5 occurs
@t=14, P3 selected for 2 units
@t=16, context switch 6 occurs
@t=17, P1 selected for 1 unit
@t=18, context switch 7 occurs
@t=19, P3 selected for 2 units
@t=21, all processes complete
Completed in 21 cycles.
```

Step-by-step metric calculations (using the formulas in Section 4):

PID	Arrival	Burst	First Start	Completion	Response	Waiting
P1	0	5	0	18	0	13
P2	1	3	3	13	2	9
P3	2	6	6	21	4	13

```
Average response time = (0 + 2 + 4) / 3 = 2.00

Average waiting time  = (13 + 9 + 13) / 3 = 11.67
```

7.5 Random (Non-preemptive)

Random (one possible outcome):

```
@t=0, P1 selected for 5 units
@t=5, context switch 1 occurs
@t=6, P3 selected for 6 units
@t=12, context switch 2 occurs
@t=13, P2 selected for 3 units
@t=16, all processes complete
Completed in 16 cycles.
```

Step-by-step metric calculations (using the formulas in Section 4):

PID	Arrival	Burst	First Start	Completion	Response	Waiting
P1	0	5	0	5	0	0
P2	1	3	13	16	12	12
P3	2	6	6	12	4	4

```
Average response time = (0 + 12 + 4) / 3 = 5.33

Average waiting time  = (0 + 12 + 4) / 3 = 5.33
```

6. A Second Worked Example (All Five Algorithms)

```
Enter random seed: 1611
Enter number of processes (2-10): 3
Enter last arrival time (0-99): 2
Enter maximum burst time (1-100): 6
Enter RR quantum (1-100): 2
Enter context-switch latency (0-10): 0
```

```
Example process set:
P1: arrival=0, burst=4
P2: arrival=1, burst=2
P3: arrival=2, burst=1
Context-switch latency L = 0
RR quantum q = 2
```

FCFS Trace:

```
@t=0 P1 runs 4
@t=4 P2 runs 2
@t=6 P3 runs 1
Completed in 7 cycles.
Avg wait = 2.33
Avg response = 2.33
```

SJF Trace:

```
@t=0 P1 runs 4
@t=4 P3 runs 1
@t=5 P2 runs 2
Completed in 7 cycles.
Avg wait = 2.00
Avg response = 2.00
```

SRTF Trace:

```
@t=0 P1 runs 1
@t=1 P2 runs 2
@t=3 P3 runs 1
@t=4 P1 runs 3
Completed in 7 cycles.
Avg wait = 1.33
Avg response = 0.33
```

RR Trace (q=2):

```
@t=0 P1 runs 2
@t=2 P2 runs 2
@t=4 P3 runs 1
@t=5 P1 runs 2
Completed in 7 cycles.
Avg wait = 2.00
Avg response = 1.00
```

Random Trace (one possible order P1,P3,P2):

```
@t=0 P1 runs 4
@t=4 P3 runs 1
@t=5 P2 runs 2
Completed in 7 cycles.
Avg wait = 2.00
Avg response = 2.00
```

7. Checksum Explanation

Checksum verifies correctness.

For each algorithm i:
Ci = completion time
W100i = round(avg_wait(i) × 100)
R100i = round(avg_response(i) × 100)

CHECKSUM = sum(Ci) + sum(W100i) + sum(R100i)

Checksum Pseudocode

```
totalCompletion = 0
totalWait100 = 0
totalResp100 = 0

for each algorithm:
    totalCompletion += completion
    totalWait100 += round(avg_wait * 100)
    totalResp100 += round(avg_response * 100)

checksum = totalCompletion + totalWait100 + totalResp100
print checksum
```

Checksum Numeric Calculation (Example):

```
Completion sum = 7+7+7+7+7 = 35
W100 sum = 233 + 200 + 133 + 200 + 200 = 966
R100 sum = 233 + 200 + 33 + 100 + 200 = 766
Final CHECKSUM = 35 + 966 + 766 = 1767
```

```
CHECKSUM: 1767
```

8. Extra Credit (+5): Edge-Case Challenge Run

Submit a FOURTH run that satisfies ALL of the following:

- Context-switch latency L > 0
- At least one idle gap occurs
- At least one tie occurs (arrival, burst, or remaining time)
- At least one preemption occurs (SRTF or RR)

In addition to the full run output, include a 5–8 sentence explanation that references specific timestamps in your trace and clearly demonstrates understanding of the scheduling behavior.

Your explanation must explicitly identify:

1) Idle Period

- State the exact timestamp when the CPU becomes idle.
- State when execution resumes.
- Identify which processes had completed.
- Identify when the next process arrives.
- Confirm that NO context-switch latency was charged during the idle jump.

2) Tie Condition

- State the timestamp of the tie.
- Identify all processes involved.
- Explain why they were equally eligible.
- Explain how the tie was resolved (RNG behavior).
- Confirm deterministic behavior with the seed.

3) Preemption Event

- State the timestamp of preemption.
- Identify the running process and the replacing process.

- Explain why preemption occurred (strictly smaller remaining time or quantum expiration).
- Confirm context-switch latency was applied.
- State when the new process begins execution.

4) Latency Handling

- Confirm latency is charged only when switching between processes.
- Confirm latency is NOT charged before the first process.
- Confirm latency is NOT charged after the final process.
- Confirm latency is NOT charged during idle jumps.

9. What to turn in

You must submit the following items:

1) Source Code
• Submit all source code files required to compile and run your program.
• Your main file must include a comment header at the top containing:
  - Course name (CS 440)
  - Assignment name (Project 3 – CPU Scheduling)
  - Names of ALL team members (if working in pairs)
  - Date submitted

If working in pairs, BOTH student names must appear in the comment header.
If only one name appears, the assignment will be treated as individual work.

Example Header (language-independent format):

```
/************************************************
 CS 440 - Operating Systems
 Project 3 - CPU Scheduling Simulation
 Names: Jane Doe, John Smith
 Date: 2026-03-11
************************************************/
```

2) Output Files (Text Format)
- You must submit THREE complete runs using three different seeds.
- Include the prompts and responses to the prompts in the files.
- Each run must be saved as a separate TEXT file (.txt).
- If completing the extra credit, submit FOUR runs.

Each run file must contain:

- The input values used
- The generated process table
- Full output for ALL FIVE algorithms
- The final CHECKSUM for that run

File naming format (example):
run1_seed10.txt
run2_seed25.txt
run3_seed42.txt
If extra credit:
run4_edgecase.txt

3) Submission Format
- Submit a single ZIP file containing:
  - All source code files
  - The required run text files (3 or 4)
- The ZIP file name should follow this format:
  LastName1_LastName2_CS440Project3.zip
  (If individual: LastName_CS440Project3.zip)

10. Grading Rubric

Input validation	8
FCFS correctness	12
SJF correctness	12
SRTF correctness	20
RR correctness	20
Random correctness	8
Metric correctness	10
Output format	5
Required runs	5
Checksum correct	10
Total	100 (+5 extra credit)

11. Common Errors

- Charging latency before first process or after final process.
- Preempting when remaining times are equal.
- Using incorrect waiting formula.
- Not re-queueing RR process.
- Forgetting to include all five algorithms in checksum.